FractalDevelopment

https://fractal-dev.xyz/

Security Audit Report

Shielded Vote SDK

Cosmos SDK v0.53 Appchain

Halo2 ZKPs · RedPallas · ElGamal · Poseidon Merkle Trees

Repository	`valargroup/vote-sdk`
Branch	`main` (commit `73d4cf82`)
Audit Window	18–27 March 2026
Status	**FINAL v1.0**

1 CRITICAL · 4 HIGH · 7 MEDIUM · 3 LOW · 10 INFO

---

13 resolved · 2 accepted risk · 0 open

10 of 10 audit areas: **SECURE**

Disclaimer

This security audit is based on the source code, documentation, and configuration available at the time of the engagement. The findings represent the auditors' best professional assessment but do not constitute a guarantee that the codebase is free of vulnerabilities. Security is a continuous process; new attack vectors, dependency vulnerabilities, or implementation changes may introduce risks not covered by this report.
The auditors accept no liability for losses arising from the use or misuse of this report or the audited software. This report does not constitute legal, financial, or investment advice.

Version	Date	Changes
v0.1	2026-03-19	Round 1 findings (18 findings across 10 areas)
v0.2	2026-03-21	Round 2 findings; F-11 added; all fix verifications
v0.3	2026-03-23	Review integration; structural hardening
v0.4	2026-03-24	E2E security testing; F-13 added
v0.5	2026-03-25	F-14 (FFI proof size DoS) added
v0.6	2026-03-25	F-15 (threshold ceremony downgrade); I-10 (curvey dependency) added; I-01, I-06 removed
v0.7	2026-03-26	F-14, F-15 resolved (PRs #43, #40); final security review
v0.8	2026-03-26	Formatting and layout improvements
v0.9	2026-03-27	F-04, F-13 resolved (PRs #44, #42); pre-release review
v1.0	2026-03-27	FINAL release

Contents

1 Executive Summary **4**
1.1 Audit Methodology . . . 4
1.1.1 Scope and Approach . . . 4
1.1.2 Techniques Used . . . 4
1.1.3 Tooling . . . 5
1.1.4 Files Reviewed . . . 5
1.2 Overall Verdict . . . 6
1.3 Findings Summary . . . 7

2 Architecture Overview **8**
2.1 System Design . . . 8
2.2 Dual-Mode Transaction Encoding . . . 8
2.3 Key Cryptographic Primitives . . . 8
2.4 State Machine . . . 9

3 Audit Areas **10**
3.1 Area 1: Cryptographic Primitives (API Layer) . . . 10
3.2 Area 2: Tally Integrity . . . 10
3.3 Area 3: ZK State Machine Gating . . . 10
3.4 Area 4: Fee Strategy . . . 10
3.5 Area 5: Alternate Tx Encoding . . . 11
3.6 Area 6: Threshold Key Ceremony . . . 11
3.7 Area 7: Message Authorization . . . 11
3.8 Area 8: Helper Server Privacy . . . 11
3.9 Area 9: Vote Tree Persistence & FFI . . . 12
3.10 Area 10: General Correctness . . . 12

4 Detailed Findings **13**
F-01 [CRITICAL] — Noop-Signer ZKP Bypass . . . 13
F-02 [HIGH] — Late MsgRevealShare Race . . . 15
F-03 [HIGH] — Malicious Dealer Retains ea_sk . . . 16
F-12 [HIGH] — Sign-Bit Tally Corruption . . . 17
F-15 [HIGH] — Threshold Ceremony Downgrade . . . 19
F-04 [MEDIUM] — TALLYING Timeout Absence . . . 21
F-06 [MEDIUM] — Zero-Cost Cosmos Txs . . . 22
F-07 [MEDIUM] — Witness Data Retention . . . 23
F-08 [MEDIUM] — Share Count Distribution Leak . . . 24
F-10 [MEDIUM] — Unused On-Chain VkZkp Fields . . . 25
F-13 [MEDIUM] — RedPallas Identity Bypass . . . 26
F-14 [MEDIUM] — Proof Size DoS via FFI . . . 28
F-05 [LOW] — Genesis Tree Root Corruption . . . 30
F-09 [LOW] — Genesis Export Gaps . . . 31
F-11 [LOW] — CommitmentLeaves Range Cap . . . 32
Informational Findings (I-01 – I-10) . . . 33

5 Trust Model and Security Boundaries **34**
5.1 Trust Roles . . . 34
5.2 Security Properties . . . 34
5.3 Compound Risk: F-03 + F-08 Privacy Degradation . . . 35

6 Codebase Maturity Evaluation **36**

7 Recommendations **37**
7.1 Immediate (Pre-Launch) 37
7.2 Short-Term 37
7.3 Long-Term 38

8 Conclusion **39**

Executive Summary

This report presents the results of a security audit of the **Shielded Vote SDK**, a Cosmos SDK (v0.53) appchain implementing private zero-knowledge voting. The system uses Halo2 zero-knowledge proofs, RedPallas signatures, ElGamal encryption with homomorphic accumulation, and Poseidon Merkle trees to enable shielded on-chain voting with threshold decryption.

The audit covered 10 areas spanning cryptographic API correctness, tally integrity, ZK state machine gating, fee strategy, transaction encoding, threshold ceremony, message authorization, helper server privacy, vote tree persistence with FFI, and general correctness. In total, **25 findings** were identified: 1 CRITICAL, 4 HIGH, 7 MEDIUM, 3 LOW, and 10 INFO.

The most severe finding was a **CRITICAL architectural vulnerability** (F-01) in the dual-mode transaction routing: six vote message types use `noopSignerFn` (zero required Cosmos signers), and four of those had no fallback authorization in their keeper handlers. A rogue validator (or any party controlling a funded on-chain account, limited to the Vote Manager or validators) could wrap these messages in standard Cosmos `Tx` envelopes, bypassing the entire ZKP/RedPallas verification pipeline. **This finding has been resolved** (PRs #15, #16 — merged 2026-03-18) with a two-layer defense: multi-message tx rejection and per-type message blocking at the ante handler. PR #17 additionally resolved F-06 by replacing unrestricted `MsgSend` with a role-based `MsgAuthorizedSend`.

The integration of cryptographic primitives, custom wire format, threshold ceremony, and Poseidon Merkle tree FFI are all correctly implemented. With the F-01 fix applied, the vote protocol's privacy and integrity guarantees hold across both transaction encoding paths.

Audit Methodology

Scope and Approach

The audit was conducted as a **full-source manual code review** of the vote-sdk codebase at commit `73d4cf82` on the `main` branch.

The scope covered the Go API surface, FFI boundary safety, input validation, and correct usage of cryptographic primitives across 10 defined audit areas. **Internal cryptographic correctness of the Halo2 circuits, RedPallas signature scheme, and Poseidon hash function is out of scope.** The audit verifies that the Go code correctly calls the Rust FFI, validates inputs/outputs at the boundary, and handles errors — but does not audit the constraint systems, witness generation, or soundness properties of the underlying ZKP circuits themselves.

Techniques Used

- **Manual code review:** Line-by-line examination of all source files in scope (∼67 Go files, 6 Rust files, 1 external Cosmos SDK file). Every function body in the audit scope was read.
- **Full code path tracing:** For each of the 11 message types, the complete path from raw bytes through decoder, ante handler, and keeper handler was traced, identifying every validation gate and state mutation.
- **Cross-area analysis:** Findings from individual areas were cross-referenced against other areas to identify compounding effects (e.g., the noop-signer bypass compounds with the TALLYING timeout absence and the late-share race condition).

- **SDK source verification:** For the CRITICAL finding, the Cosmos SDK v0.53.5 source code (`x/auth/ante/sigverify.go`) was independently analyzed to confirm the zero-signer bypass at the SDK level.
- **Mathematical verification:** Threshold formulas, half-ack conditions, sampling distributions, and Lagrange interpolation invariants were verified algebraically.
- **Grep-based sweeps:** Systematic searches for anti-patterns (`_ = err`, direct keeper calls outside handlers, authz/governance/sudo modules) across the codebase.
- **End-to-end functional verification:** The complete voting lifecycle was executed on a live test node — session creation, key ceremony (deal, ack), voter delegation, vote casting, share reveal, tally, and decryption — with each step verified for correctness against expected on-chain state transitions. This dynamic testing confirmed that the system behaves as specified under normal operation and surfaced edge cases not visible through static analysis alone.
- **Chaos testing:** Adversarial traffic was injected against a live 3-validator testnet to validate robustness under hostile conditions. Tests included: 200 concurrent 500 KB garbage proofs targeting the Halo2 FFI (F-14), vote and ceremony messages routed through the wrong transaction encoding path (F-01), malicious deal payloads with downgraded threshold values (F-15), identity-point RedPallas keys in delegation and vote cast messages (F-13), and sustained multi-wave request floods to measure latency degradation under load. All payloads were rejected gracefully — no crashes, no OOM, no app-hash divergence across validators.
- **Coverage gap audit:** A dedicated pass identified all code paths not covered in the primary audit rounds, including CLI key generation commands, helper server proof pipeline, consensus-level PrepareProposal/ProcessProposal attack vectors, and module registration. No new HIGH or MEDIUM findings were identified. All unreviewed paths use `crypto/rand`, proper file permissions (0600 for secrets), and consistent input validation.

Tooling

Automated static analysis was performed using `go vet` (Go 1.25.7), `staticcheck` (v0.7.0), `golangci-lint` (v1.64.8), `govulncheck` (v1.1.4, DB 2026-03-23), `cargo audit` (v0.22.1, 985 advisories), and `cargo clippy` (Rust 1.92.0). Six targeted anti-pattern sweeps were run against all production Go source files. No vulnerabilities, no dependency advisories, and zero anti-pattern matches were found. `staticcheck` reported 2 cosmetic production warnings; `golangci-lint` reported 3 unchecked JSON encode errors in the off-chain helper server. No security-relevant issues were identified by any automated tool.

Files Reviewed

A total of **74 source files** were reviewed (excluding generated `.pb.go`), covering:

- `app/` — Ante handlers, proposal injection, decoder, module wiring (11 files)
- `x/vote/` — Keeper, handlers, types, ante pipeline, module (26 files)
- `ffi/` — Vote tree FFI, ZKP bridge, RedPallas bridge (18 files)
- `api/` — Wire format codec, tx wrapper (5 files)
- `internal/helper/` — Processor, store, API, types (7 files)
- Rust sources — `ffi.rs`, `votetree.rs`, `redpallas.rs`, `nc_root.rs`, `lib.rs`, `toy.rs` (6 files)
- External: Cosmos SDK v0.53.5 `x/auth/ante/sigverify.go` (1 file)

Overall Verdict

#	Area	Verdict
1	Cryptographic Primitives (API Layer)	**SECURE**
2	Tally Integrity	**SECURE**†
3	ZK State Machine Gating	**SECURE**†
4	Fee Strategy	**SECURE**
5	Alternate Tx Encoding	**SECURE**
6	Threshold Key Ceremony	**SECURE**
7	Message Authorization	**SECURE**†
8	Helper Server Privacy	**SECURE**
9	Vote Tree Persistence & FFI	**SECURE**
10	General Correctness	**SECURE**

†Areas 2, 3, and 7 were originally *Conditional Pass* pending remediation of F-02/F-12 (Area 2) and F-01 (Areas 3, 7). All three findings have been resolved; these areas now pass unconditionally.

Findings Summary

ID	Severity	Title	Area	Status
F-01	CRITICAL	Noop-signer vote messages bypass ZKP validation via standard Cosmos Tx	7	✓
F-02	HIGH	Late `MsgRevealShare` during TALLYING invalidates stored partial decryptions	2	✓
F-03	HIGH	Malicious dealer retains full `ea_sk`	6	Accepted
F-12	HIGH	ZKP #3 `enc_share` sign-bit ambiguity allows silent tally corruption	2	✓
F-15	HIGH	Proposer can downgrade threshold ceremony to single-share security	6	✓
F-04	MEDIUM	TALLYING state has no timeout or recovery path	6	✓
F-06	MEDIUM	Validators can send unrestricted Cosmos txs at zero cost	4	✓
F-07	MEDIUM	Helper server retains witness data after proof generation	8	✓
F-08	MEDIUM	Share counts leak vote distribution before finalization	2	Accepted
F-10	MEDIUM	On-chain `VkZkp1/2/3` unused — circuits not upgradeable without hard fork	3	✓
F-13	MEDIUM	RedPallas identity-point signature bypass — four validation layers non-functional	7	✓
F-14	MEDIUM	Unbounded proof size in `MsgRevealShare` enables CPU DoS via FFI	3, 4	✓
F-05	LOW	Genesis import produces incorrect tree root	10	✓
F-09	LOW	Genesis export omits partial decryptions and block leaf index	9	✓
F-11	LOW	`CommitmentLeaves` query has no range cap	–	✓
I-01	INFO	Non-constant-time point comparison in tally verification	9	Accepted
I-02	INFO	Slice aliasing in `StripNonAckersFromRound`	6	Accepted
I-03	INFO	Missing field size checks in `ValidateBasic`	3	✓
I-04	INFO	RedPallas `rk` not validated as Pallas point before FFI	3	✓
I-05	INFO	`UpdateVoteRoundStatus` lacks transition guard	9	✓
I-06	INFO	Missing `catch_unwind` on 18 of 19 Rust FFI functions	–	✓
I-07	INFO	Ack "signature" is SHA256 hash, not cryptographic signature	6	✓
I-08	INFO	Redundant partial decryption injection when tally threshold met	–	Accepted
I-09	INFO	Share file accumulation without cleanup (Shamir scalars on disk)	–	✓
I-10	INFO	Sole Pallas curve dependency (`curvey`) is dormant single-maintainer library	1	Accepted

Architecture Overview

System Design

The Shielded Vote SDK is a single-purpose Cosmos appchain where:

- A **vote manager** (trusted role) creates voting sessions with cryptographic parameters.
- **Voters** submit shielded votes authenticated by Halo2 zero-knowledge proofs and RedPallas signatures.
- **Validators** participate in a threshold key ceremony to distribute the ElGamal election authority key.
- **Tallying** uses ElGamal homomorphic accumulation on-chain, verified via DLEQ proofs and threshold Lagrange combination.

Dual-Mode Transaction Encoding

The chain uses two transaction formats:

1. **Standard Cosmos `Tx` envelope** — For administrative messages (`MsgCreateVotingSession`, `MsgSetVoteManager`, `MsgRegisterPallasKey`, etc.). These go through the standard Cosmos ante handler with signature verification.
2. **Custom 1-byte-tag wire format** — `[tag][protobuf payload]`. Used for vote messages (tags `0x02--0x05`) and auto-injected ceremony messages (tags `0x07`, `0x08`, `0x0D`). Tag `0x0A` is deliberately excluded to avoid collision with Cosmos Tx protobuf field 1 (wire type 2).

The routing decision is in `app/ante.go:73--76`: if the decoded transaction is a `*VoteTxWrapper`, it is routed to the custom ZKP/signature validation pipeline; otherwise, it goes through the standard Cosmos ante chain.

Key Cryptographic Primitives

Primitive	Usage
Halo2 (Rust FFI)	Three ZKP circuits: delegation proof, vote commitment proof, share reveal proof
RedPallas (Rust FFI)	Spend authorization signatures for delegation and vote cast messages
ElGamal on Pallas	Homomorphic encryption of vote shares; DLEQ proof for tally verification
Poseidon Merkle Tree	Depth-24 commitment tree for note commitments (Rust FFI)
Shamir Secret Sharing	(t, n) threshold splitting of `ea_sk` with $t = \max(2, \lceil n/2 \rceil)$
ECIES (ChaCha20-Poly1305)	Encryption of Shamir shares to validator Pallas public keys
BLAKE2b-256	Fiat–Shamir challenge hash (domain `"svote-dleq-v1"`) and sighash computation

State Machine

Round lifecycle (irreversible):

$$\texttt{PENDING} \longrightarrow \texttt{ACTIVE} \longrightarrow \texttt{TALLYING} \longrightarrow \texttt{FINALIZED}$$

Ceremony lifecycle (with timeout reset):

$$\texttt{REGISTERING} \longrightarrow \texttt{DEALT} \longrightarrow \texttt{CONFIRMED}$$

$$\texttt{DEALT} \xrightarrow{\text{timeout, insufficient acks}} \texttt{REGISTERING}$$

All session status transitions are strictly unidirectional. The ceremony reset (DEALT → REGISTERING) only affects `CeremonyStatus` and never regresses `SessionStatus`.

Audit Areas

Area 1: Cryptographic Primitives (API Layer)

Verdict: **SECURE**

Scope: Go API surface in `crypto/elgamal/`, `crypto/shamir/`, `crypto/ecies/`, `ffi/votetree/`, and the FFI bridge. Input validation, serialization, error handling.

Key findings: All API-layer checks pass. `UnmarshalPublicKey` rejects the identity point. `LagrangeCoefficients` rejects duplicate and ≤ 0 indices. ECIES uses fresh ephemeral keys per encryption with proper ECDH degenerate-point rejection. Shamir uses 1-based indices. DLEQ uses two distinct domain tags (`"svote-dleq-v1"` and `"svote-pd-dleq-v1"`).

Area 2: Tally Integrity

Verdict: **SECURE**

Scope: Full tally pipeline: ElGamal homomorphic accumulation, threshold Lagrange combination, DLEQ verification, tally completeness, BSGS recovery.

Key findings: DLEQ is verified per partial-decryption submission (not deferred to tally time). `ValidateTallyCompleteness` ensures all (proposal, decision) pairs are present. Lagrange interpolation uses the correct original ShamirIndex values, surviving validator stripping. Finding F-02 (late share race condition) and **Finding F-12** (sign-bit tally corruption) have been **resolved**. **Finding F-08** (share count leak) still affects this area.

Area 3: ZK State Machine Gating

Verdict: **SECURE**

Scope: Ante pipeline enforcement, validation ordering, absence of governance/sudo bypasses, deterministic EndBlocker transitions.

Key findings: The custom wire format path correctly gates all messages. Validation ordering is cheapest-first (ValidateBasic → round check → nullifiers → RecheckTx return → ZKP/sig). Nullifiers are always checked, even on RecheckTx. No authz, governance, or sudo modules detected. Finding F-01 (noop-signer bypass via standard Cosmos Tx) has been **resolved** — vote/ceremony messages are now rejected at the ante handler with defense-in-depth multi-message blocking. Findings F-10 (unused on-chain VkZkp fields) and F-14 (unbounded proof size DoS) have also been **resolved**.

Area 4: Fee Strategy

Verdict: **SECURE**

Scope: Fee-free chain design safety, spam resistance mechanisms.

Key findings: Vote protocol messages are protected by nullifier uniqueness, proposer-only injection, validator gating, one-pending-round limit, and VoteManager-only session creation. Finding F-06 (unrestricted zero-cost Cosmos txs) has been **resolved** — `MsgSend`/`MsgMultiSend` are now blocked, replaced by role-based `MsgAuthorizedSend`.

Area 5: Alternate Tx Encoding

Verdict: **SECURE**

Scope: Custom wire format, tag collision prevention, three-layer proposer injection.

Key findings: Tag 0x0A correctly excluded. Decoder routing is deterministic. Three-layer proposer injection (mempool barrier, ProcessProposal, FinalizeBlock) is sound. All four `validateInjected*` functions verify proposer identity.

Area 6: Threshold Key Ceremony

Verdict: **SECURE**

Scope: Key generation, Shamir splitting, ECIES distribution, ack verification, half-ack evaluation, timeout recovery, dealer trust model.

Key findings: Fresh `ea_sk` per ceremony with proper zeroing. ECIES uses fresh ephemeral keys with ChaCha20-Poly1305 (zero nonce safe by unique-key construction). Feldman VK verification on ack (`share_i * G == VK_i`). Half-ack formula (`acks*2 >= n`) correctly implements acks $\geq \lceil n/2 \rceil$. ShamirIndex preserved through stripping. **Finding F-03** (malicious dealer) affects vote confidentiality. **Finding F-04** (no TALLYING timeout) has been **resolved**. **Finding F-15** (threshold ceremony downgrade) has been **resolved**.

Area 7: Message Authorization

Verdict: **SECURE**

Scope: Complete auth matrix for all 11 message types, domain separation, validator spam.

Key findings: All 11 message types have their stated authorization gates when accessed via the intended code path. Domain separation (ZIP-244 vs Blake2b-256 vs SHA-256 ack hash) is structurally sound. Finding F-01 (noop-signer bypass) has been **resolved** — the ante handler now rejects vote/ceremony messages in standard Cosmos txs and blocks multi-message txs entirely. **Finding F-13** (RedPallas identity-point bypass) has also been **resolved**.

Area 8: Helper Server Privacy

Verdict: **SECURE**

Scope: Timing privacy, SQLite share lifecycle, proof generation error handling.

Key findings: `exponentialDelay()` correctly samples Exp(1/mean) via `crypto/rand` with standard inverse CDF. Crash recovery re-randomizes all delays. Error messages leak no witness data. `ON CONFLICT DO NOTHING` prevents share overwrite. API uses constant-time token comparison and 1MB body limit. **Finding F-07** (witness data retention) affects post-proof privacy.

Area 9: Vote Tree Persistence & FFI

Verdict: **SECURE**

Scope: Shard round-trip fidelity, cold start, determinism, FFI boundary safety, root soundness as ZKP input.

Key findings: `cgo.Handle` lifecycle is correct (Rust free → Go handle delete → ctxPtr free). Buffer ownership is clean with single-transfer protocol. No floating-point, no HashMap, pure Fp arithmetic ensures determinism. Leaf validation rejects malformed/non-canonical inputs. Close() is idempotent. **Finding F-05** (genesis import tree root corruption) affects chain migration scenarios.

Area 10: General Correctness

Verdict: **SECURE**

Scope: State machine transitions, integer safety, error propagation, Cosmos SDK gotchas, genesis completeness.

Key findings: All session status transitions are unidirectional. Zero `_ = err` patterns in production code. MaxProposals=15 and MaxVoteOptions=8 enforced at both ValidateBasic and handler levels. EndBlocker uses collect-then-mutate pattern. No non-deterministic iteration. **Finding F-09** (genesis prefix omission) is LOW severity.

Detailed Findings

CRITICAL

F-01: Noop-Signer Vote Messages Bypass ZKP Validation via Standard Cosmos Tx Envelope

Threat model: Single rogue validator (1 of N).
Affected code:

- `x/vote/module.go:82--84` (`noopSignerFn`)
- `app/ante.go:73--98` (dual-mode routing)
- `x/vote/keeper/msg_server.go:145--217` (keeper handlers)

Description

The chain uses a dual-mode transaction encoding: vote and ceremony messages are intended to be submitted exclusively via the custom 1-byte-tag wire format, which routes them through the ZKP/RedPallas validation pipeline. These message types use `noopSignerFn` (returning zero required Cosmos signers) because their authentication is handled by the custom ante chain, not by Cosmos signature verification. This is an intentional design — the keeper handlers rely on the upstream custom pipeline for authorization rather than duplicating checks internally.

The vulnerability arises from the interaction between this design and the standard Cosmos encoding path. All six vote/ceremony message types are registered as `sdk.Msg` via `RegisterInterfaces`, making them structurally valid for inclusion in standard Cosmos `TxRaw` envelopes. A rogue validator could construct a standard `TxRaw` containing a noop-signer vote message (either as a multi-message bundle or by forcing the standard encoding path), bypassing the custom wire format entirely. The `CustomTxDecoder` routes it to the standard Cosmos decoder (first byte `0x0A`, not a custom tag). The `DualAnteHandler` routes it to the standard ante chain. The `SigVerificationDecorator` in Cosmos SDK v0.53.5 (`sigverify.go:332--335`) checks `len(sigs) != len(signers)` — for zero signers, this is `0 != 0 = false`, so the check passes. The verification loop at line 346 never executes.

Note: This is only exploitable by validators on the network. External parties cannot submit transactions.

Impact

- `MsgRevealShare`: Arbitrary ciphertexts added to the homomorphic accumulator via `AddToTally` — **complete tally corruption**.
- `MsgDelegateVote`: Arbitrary commitment tree pollution and governance nullifier DoS.
- `MsgCastVote`: Fake vote commitments and VAN nullifier DoS.
- `MsgSubmitTally`: A rogue validator can finalize a round (math still verified, lower severity).

This finding compounds with F-02 (late shares invalidate partial decryptions) and F-04 (no TALLYING timeout): a corrupted round becomes permanently unrecoverable.

Recommendation

Add an ante decorator to the standard chain that rejects any message type intended for the custom wire format:

```
case *types.MsgDelegateVote, *types.MsgCastVote,
     *types.MsgRevealShare, *types.MsgSubmitTally,
     *types.MsgAckExecutiveAuthorityKey,
     *types.MsgSubmitPartialDecryption,
     *types.MsgDealExecutiveAuthorityKey:
    return fmt.Errorf("message␣%T␣must␣use␣custom␣wire␣format", msg)
```

Additionally, add handler-level authorization checks as defense-in-depth.

Remediation

Status: RESOLVED (PRs #15, #16, #17 — merged 2026-03-18/19).

The fix implements four independent defense layers in `app/ante.go`, any one of which is sufficient to block the bypass:

1. **Multi-message rejection (PR #16):** Standard Cosmos txs with more than one message are rejected outright at `app/ante.go:81--83`. This eliminates the entire noop-signer carrier attack class — no legitimate message can be bundled with a noop-signer message.
2. **Explicit type check (PR #15):** An `isVoteModuleMsg` check at `app/ante.go:86--89` rejects all 7 vote/ceremony message types in standard Cosmos txs, even as single messages.
3. **Positive-security allowlist (PR #15):** A `MessageWhitelistDecorator` at `app/ante_whitelist.go` rejects any message whose proto type URL is not in a hardcoded set of 8 allowed types. Vote/ceremony messages are absent — a deny-by-default model where future types are blocked unless explicitly allowlisted.
4. **Signature verification:** All 8 allowlisted messages have proper signer functions (not noop), so `SigVerificationDecorator` requires a valid Cosmos signature. No noop-signer message can reach handler execution via the standard path.

PR #17 additionally blocks `MsgSend` and `MsgMultiSend` at the ante handler, replacing them with a role-based `MsgAuthorizedSend` (see F-06 remediation).

Bypass vectors verified as blocked: standard `TxRaw` wrapping, `authz.MsgExec` wrapping (x/authz not registered), direct gRPC (CometBFT only accepts txs via ABCI), Amino encoding (same decoder path), and future unknown message types (whitelist rejects by default).

Test coverage includes 27 ante handler tests: table-driven rejection of all blocked types, noop-signer POCs for all 7 message types, tally corruption repro, zero-cost attack POC, authz `MsgExec` bypass test, and whitelist allowlist/blocklist tests.

HIGH

F-02: Late MsgRevealShare During TALLYING Invalidates Stored Partial Decryptions

Threat model: Accidental (normal usage) or deliberate griefing.
Affected code:

- `x/vote/types/msgs.go:265`
- `x/vote/keeper/msg_server_tally_decrypt.go:189--292`
- `x/vote/keeper/keeper_tally.go:332--346`

Description

`MsgRevealShare.AcceptsTallyingRound()` returns `true`, and `ValidateRoundForShares` accepts shares unconditionally during TALLYING. This means a `MsgRevealShare` can be submitted and included in blocks while partial decryptions are being collected.

Each `MsgRevealShare` modifies the tally accumulator (changing both C_1 and C_2). A partial decryption $D_i = \text{share}_i \times C_1$ is bound to the specific C_1 value at the time of computation. If the accumulator changes after D_i is stored but before all threshold partials are collected, the Lagrange combination produces an incorrect result: $\lambda_A \cdot D_A + \lambda_B \cdot D_B = \lambda_A \cdot (\text{share}_A \cdot C_1^{\text{old}}) + \lambda_B \cdot (\text{share}_B \cdot C_1^{\text{new}})$, which does not equal $\text{ea_sk} \cdot C_1^{\text{new}}$.

The duplicate partial check (`msg_server_tally_decrypt.go:222--229`) prevents resubmission, so the stale D_A is permanent. Combined with F-04 (no TALLYING timeout), the round is stuck forever.

Recommendation

Change `MsgRevealShare.AcceptsTallyingRound()` to return `false`. Shares submitted after the ACTIVE → TALLYING transition would be rejected at the ante layer.

Remediation

Status: RESOLVED (PR #21 — merged 2026-03-19).

The fix implements three layers of defense:

1. **Ante handler gate:** `AcceptsTallyingRound()` now returns `false` for `MsgRevealShare`. All vote messages route through `ValidateRoundForVoting`, which rejects non-ACTIVE rounds. The permissive `ValidateRoundForShares` function was deleted as dead code.
2. **Handler-level defense-in-depth:** A `ValidateRoundForVoting` check was added at the top of the `RevealShare` handler in `msg_server_tally_decrypt.go`, protecting against messages injected via `PrepareProposal` that bypass the ante handler.
3. **Helper optimization:** A `RoundStatusChecker` skips ZKP computation for shares targeting non-ACTIVE rounds, avoiding wasted work.

Since the ACTIVE → TALLYING transition occurs in EndBlocker (after all messages in the block have been processed), there is no window where a `RevealShare` could execute in the same block as partial decryptions.

HIGH

F-03: Malicious Dealer Retains Full ea_sk

Threat model: Single rogue validator (1 of N).
Affected code: `app/prepare_proposal_ceremony.go:148--151`.

Description

The dealer generates `ea_sk` at `prepare_proposal_ceremony.go:148` and retains it in memory through the deal construction. Although `defer zeroScalar(eaSk.Scalar)` runs at function exit, a modified binary can exfiltrate the key before zeroing. With `ea_sk`, the attacker decrypts every individual vote: $v \cdot G = C_2 - \texttt{ea_sk} \cdot C_1$, then $v = \text{BSGS}(v \cdot G)$.

The attack is invisible to all on-chain validation. With n validators and round-robin proposer selection, the rogue validator deals approximately $1/n$ of voting rounds.

Recommendation

Replace the single-dealer model with a DKG protocol where each validator contributes randomness to `ea_sk`, ensuring no single party knows the full key.

Remediation

Status: ACCEPTED RISK. The client has elected not to fix this finding. The single-dealer model is an intentional design trade-off: implementing a full DKG protocol adds significant complexity and is deferred to a future version. The risk is mitigated by the permissioned validator set (all validators are known and trusted to a degree) and round-robin dealer rotation (any single rogue validator only deals $\sim 1/n$ of rounds). Vote *integrity* is unaffected — only vote *confidentiality* for rounds dealt by the rogue validator is at risk.

HIGH

F-12: ZKP #3 enc_share Sign-Bit Ambiguity Allows Silent Tally Corruption

Threat model: Any voter with a valid Vote Authority Note (legitimate stakeholder).
Affected code:

- `ffi/zkp/halo2/verify.go:399--405` (sign-bit clearing)
- `x/vote/keeper/keeper_tally.go:49--64` (accumulation with original bytes)
- `voting-circuits: share_reveal/circuit.rs:94--113` (x-coordinate-only public inputs)

Description

The share reveal circuit (ZKP #3) exposes only the x-coordinates of the ElGamal ciphertext points C_1 and C_2 as public inputs (slots 1 and 2). The Go verifier clears sign bits at `verify.go:403,405` (`buf[63] &= 0x7F`, `buf[95] &= 0x7F`) before passing x-coordinates to the Rust FFI. The `AddToTally` handler at `keeper_tally.go:40` uses the original `msg.EncShare` bytes (sign bits intact) via `UnmarshalCiphertext`.

For any valid x-coordinate on the Pallas curve, two compressed representations exist: (x, y) and $(x, -y)$, differing only in the sign bit of byte 31. Both pass `UnmarshalCiphertext` and produce the same x-coordinate after sign-bit clearing. The ZKP verifies against x-coordinates and passes for either variant.

Attack: A malicious voter generates a valid proof for honest (C_1, C_2), then flips sign bits: `encShare[31] ^= 0x80; encShare[63] ^= 0x80`. The ZKP passes (same x-coordinates). `AddToTally` accumulates $(-C_1, -C_2)$. At threshold decryption:

$$-C_2 - \mathrm{sk} \cdot (-C_1) = -(v \cdot G + r \cdot \mathrm{ea_pk}) + \mathrm{sk} \cdot r \cdot G = -v \cdot G$$

BSGS searches $[0, 2^{28})$ and either fails (round stuck) or, when other honest voters exist in the same bin, silently returns the wrong total ($v_{\text{honest}} - v_{\text{attacker}}$ instead of $v_{\text{honest}} + v_{\text{attacker}}$).

Impact

- **Silent tally corruption:** When other honest voters exist in the same accumulator bin, the tally is wrong by $2 \cdot v_{\text{attacker}}$ — BSGS succeeds, `SubmitTally` verification passes, and the election result is silently incorrect.
- **Round locking:** When the attacker is the sole voter in the bin, BSGS fails and the round is stuck at TALLYING (compounding with F-04).
- **Low barrier:** Requires a valid ZKP proof (legitimate stakeholder) and a 1-byte XOR on the client.
- **Undetectable:** The silent corruption variant is cryptographically indistinguishable from a legitimate vote.

Go-level fixes are **insufficient**: a round-trip check is an identity operation for valid points, and clearing sign bits for both paths independently canonicalizes y-coordinates, breaking the ElGamal relationship for ~50% of honest votes.

Recommendation

Expose both (x, y) coordinates of C_1 and C_2 as public inputs in the share reveal circuit ($7 \to 9$ public inputs), matching the pattern already used for `rk` in ZKP #1 and `r_vpk`/`ea_pk` in ZKP #2. The Go verifier would decompress `msg.EncShare` via `UnmarshalCiphertext`, extract all four

coordinates, and serialize them into the FFI buffer. This is the only correct fix — the ZKP must bind the full curve point, not just the x-coordinate.

Remediation

Status: RESOLVED (PR #26 — merged 2026-03-19).

PR #19 confirmed the vulnerability with a real Halo2 proof reproduction test:

- `TestSignBitFlip_RealProofVerifiesBothEncodings` — demonstrates that flipped sign bits pass the original verifier.
- PR #26 implements the recommended fix: the share reveal circuit now includes y-coordinates of C_1 and C_2 as public inputs ($7 \to 9$ inputs).
- The Go verifier decompresses `msg.EncShare` via `UnmarshalCiphertext` → `ToAffineUncompressed`, extracting full (x, y) coordinates and replacing sign-bit stripping.
- Regression test `TestSignBitFlip_RejectedAfterFix` confirms that flipped `EncShare` is now rejected by the ZKP verifier.

HIGH

F-15: Proposer Can Downgrade Threshold Ceremony to Single-Share Security

Threat model: 1-of-N rogue validator (malicious block proposer / ceremony dealer).
Affected code:

- `x/vote/keeper/msg_server_ceremony.go`
- `app/process_proposal.go`
- `app/prepare_proposal_ceremony.go`
- `crypto/shamir/shamir.go`

Confirmed: Unit tests pass on current codebase proving the full attack chain.

Description

The intended threshold policy is encoded in `thresholdForN(n)` at `prepare_proposal_ceremony.go:45--57`: for $n \geq 2$, the threshold is $t = \max(2, \lceil n/2 \rceil)$. This policy is security-critical — it ensures no single validator learns the full election authority secret key (`ea_sk`) and that tally completion requires t cooperating validators.

However, the on-chain validation only checks `msg.Threshold < 1` (`msg_server_ceremony.go:117`). The `uint32` proto field means only the value 0 is rejected. A malicious proposer running a modified binary can construct a `MsgDealExecutiveAuthorityKey` with `Threshold = 1` and $n \geq 2$, and both `ProcessProposal` (which performs zero threshold validation) and the keeper handler will accept it.

With $t = 1$, `shamir.Split(ea_sk, 1, n)` produces a degree-0 polynomial where *every share equals `ea_sk`*. The Feldman verification check (`share_i * G == VK_i`) still passes because all shares and verification keys are identical. The stored `round.Threshold = 1` then allows tally completion with a single partial decryption.

The proto definitions document `threshold` as "Always ≥ 2" (`tx.pb.go:910`, `types.pb.go:287`), confirming the intended invariant. The on-chain code does not enforce it.

Attack Chain

1. Round created with $n = 3$ validators. Honest threshold: $t = 2$.
2. Malicious proposer constructs deal with `Threshold = 1`.
3. `ProcessProposal` on honest validators: passes (no threshold check).
4. `DealExecutiveAuthorityKey` handler: passes ($1 \geq 1$).
5. `shamir.Split(ea_sk, 1, 3)`: produces 3 shares, all equal to `ea_sk`.
6. Each validator acks: decrypts share, checks `share * G == VK_i` — passes (all identical).
7. All 3 validators now hold `ea_sk` (in memory and on disk via share files).
8. During TALLYING: `PrepareProposal` checks `validatorCount >= round.Threshold = 1`. One partial decryption triggers tally.
9. `CombinePartials(partials, 1)`: Lagrange coefficient $= 1$, result $= D_i = \text{ea_sk} \cdot C_1$. Correct decryption.

Impact

Dimension	Assessment
Privacy	All n validators receive `ea_sk`; any single validator can decrypt every vote
Tally trust	1-of-n instead of $\lceil n/2 \rceil$-of-n; single validator can force finalization
Detectability	Silent. Non-consensus-breaking. All honest nodes store the same state
Barrier	Attacker must be block proposer (round-robin, $\sim 1/n$ rounds)
Blast radius	Per-round (only affects rounds where attacker is dealing proposer)

This finding is **strictly worse** than F-03 (accepted risk, dealer retains `ea_sk`): F-03 leaks the secret to 1 party (the dealer); this attack leaks it to all n validators and reduces tally liveness from t-of-n to 1-of-n.

Recommendation

Enforce the threshold policy on-chain:

1. Require `msg.Threshold = thresholdForN(len(round.CeremonyValidators))` in both `validateInjectedDeal` and the `DealExecutiveAuthorityKey` keeper handler.
2. Add upper-bound check: reject `msg.Threshold > nValidators` to prevent liveness DoS via inflated threshold.
3. Add regression test for a malicious deal with $n \geq 2$ and `Threshold = 1`, asserting both `ProcessProposal` and the keeper reject it.

Remediation

Status: RESOLVED (PR #40 — merged 2026-03-26).

PR #40 enforces `ThresholdForN(n)` as an exact match in both the keeper handler (`DealExecutiveAuthorityKey`) and `ProcessProposal`'s `validateInjectedDeal`. The computation is extracted to an exported function with error handling.

Tests cover three layers:

- 14-case table-driven unit tests plus an invariant sweep ($n = 1..50$) for `ThresholdForN`
- Handler rejection tests and a full cryptographic attack chain (Shamir split, ECIES encrypt, deal submission) confirming rejection
- ABCI integration tests verifying `ProcessProposal` rejects deals with bad thresholds (6 sub-tests) while the honest `PrepareProposal` → `ProcessProposal` pipeline succeeds for $n = 2, 3, 5, 7$

MEDIUM

F-04: TALLYING State Has No Timeout or Recovery Path

Threat model: Validator liveness failure.
Affected code: `x/vote/module.go:384--577` (EndBlocker).

Description

The EndBlocker handles three state transitions: (1) tree root computation, (2) ACTIVE → TALLYING on `vote_end_time`, (3) DEALT ceremony timeout. There is **no fourth section** for TALLYING timeout. If fewer than t validators submit partial decryptions (due to crash, key file loss, or jailing), the round remains in TALLYING indefinitely.

For $n = 2$, $t = 2$ (both required). A single validator failure permanently locks the round. System liveness is unaffected (TALLYING rounds do not block new round creation), but the stuck round's votes are permanently lost.

Recommendation

Add a configurable TALLYING timeout. On expiry: if $\geq t$ partials exist, proceed with tally; otherwise, mark the round as FAILED/CANCELLED.

Remediation

Status: RESOLVED (PR #44 — merged 2026-03-27).

PR #44 adds a configurable tally phase timeout following the existing ceremony DEALT timeout pattern:

1. On the ACTIVE → TALLYING transition, the EndBlocker stores `tally_phase_start` (current block time) and `tally_phase_timeout` (default 6 hours) on the `VoteRound` proto.
2. A new fourth section in the EndBlocker checks all TALLYING rounds against their deadline. On expiry, the round transitions to FINALIZED with `tally_timed_out = true`.
3. Timed-out rounds reuse the existing `FINALIZED` status (no new enum value), preserving downstream client compatibility. The `tally_timed_out` boolean distinguishes normal finalization from timeout.
4. A zero timeout value disables the feature, preserving the previous behavior for rounds that should wait indefinitely.

Test coverage includes 5 new EndBlocker tests: timeout expiry, pre-deadline retention, multi-round independent deadlines, zero-timeout disable, and field-setting verification on ACTIVE → TALLYING.

MEDIUM

F-06: Validators Can Send Unrestricted Standard Cosmos Txs at Zero Cost

Threat model: Single rogue validator.
Affected code: `app/ante_ceremony.go:75--84`, `app/ante.go:166--183`.

Description

`CeremonyFeeExemptDecorator` grants infinite gas to ALL standard Cosmos transactions. No `DeductFeeDecorator` is present. With the bank module enabled, validators can submit `MsgSend`, staking operations, and `MsgUnjail` at zero cost with no rate limiting.

Recommendation

Add a message-type allowlist decorator restricting standard Cosmos tx messages to ceremony-related and essential operations only.

Remediation

Status: RESOLVED (PR #17 — merged 2026-03-19).

Bank `MsgSend` and `MsgMultiSend` are now blocked at the ante handler. A new `MsgAuthorizedSend` in the vote module replaces them with role-based authorization:

- The **vote manager** can send to any address (stake distribution to new validators).
- **Bonded validators** can only send to the vote manager or other bonded validators.
- All other senders are rejected.

This eliminates unrestricted zero-cost transfers while preserving the operational token flows required by the permissioned validator set. 14 unit tests and 2 integration tests cover all authorization paths.

MEDIUM

F-07: Helper Server Retains Witness Data After Proof Generation

Threat model: Local attacker with filesystem access to helper server.
Affected code: `internal/helper/store.go:443--455`.

Description

`MarkSubmitted()` sets `state = 2` but does not clear `primary_blind` or `share_comms`. Witness data persists in SQLite until round expiry. No `PRAGMA secure_delete` is enabled, allowing forensic recovery of deleted rows.

Recommendation

Clear witness columns on submission. Enable `PRAGMA secure_delete=ON`.

Remediation

Status: RESOLVED (PR #46 — merged 2026-03-27).

PR #46 implements both recommendations:

- **Witness scrubbing:** `MarkSubmitted()` and `MarkFailed()` (permanent failure path) now zero out `enc_share_c1`, `enc_share_c2`, `share_comms`, and `primary_blind` atomically with the state transition. Retry transitions are left untouched since the payload is still needed for resubmission.
- **Secure delete:** `PRAGMA secure_delete=ON` is enabled at database open, overwriting freed pages with zeros to prevent forensic recovery.

Tests assert all four witness columns are empty after submission and after permanent failure. 46 helper tests pass.

MEDIUM

F-08: Share Counts Leak Vote Distribution Before Finalization

Threat model: Any observer with node access (block indexer, RPC client, event subscriber).
Affected code:

- `x/vote/keeper/msg_server_tally_decrypt.go:44--47` — KV share count increment
- `x/vote/keeper/msg_server_tally_decrypt.go:49--55` — `reveal_share` event emission
- `x/vote/keeper/keeper_tally.go:151--208` — `VoteSummary` query exposes `BallotCount`

Description

Per-option vote distribution leaks through three independent channels during an active round, undermining the privacy guarantee of ElGamal encryption:

Channel 1 — KV share counts (original finding). Every `MsgRevealShare` increments a plaintext counter at `0x0B || round_id || proposal_id || decision`. These counters are readable via raw ABCI queries (`store/vote/key`).

Channel 2 — `reveal_share` event attributes (Round 2). The `EventTypeRevealShare` event emits `proposal_id`, `vote_decision`, and `share_nullifier` for every individual share reveal (line 54). A block indexer or event subscriber can reconstruct per-option vote counts in real time by tallying these events. The `share_nullifier` attribute adds a unique per-reveal identifier to the public event stream that serves no client-side purpose; if the nullifier derivation were ever found to be weak, it becomes a per-vote correlation vector.

Channel 3 — `VoteSummary` gRPC query (Round 2). The `VoteSummary` query handler reads the `0x0B` counters and returns per-option `BallotCount` in the `OptionSummary` proto. This is the gRPC-accessible surface of Channel 1 — any RPC client can poll it during an active round to observe vote distribution changes.

Recommendation

All three channels should be addressed together:

1. Remove per-decision share count tracking during active rounds; defer breakdown to finalization time.
2. Strip `proposal_id`, `vote_decision`, and `share_nullifier` from the `reveal_share` event. Emit only `vote_round_id` as a generic "a share was revealed" signal.
3. Gate the `VoteSummary` query's `BallotCount` field: return zero for non-finalized rounds, or omit the field entirely until `SESSION_STATUS_FINALIZED`.

Remediation

Status: ACCEPTED RISK. The client has elected not to fix this finding. The vote distribution leak is considered an acceptable trade-off given the permissioned validator set and the operational utility of real-time share counts for monitoring. The share counts serve as a liveness indicator during active rounds, and the voter population is already known to validators. Full vote-distribution privacy would require deferring all per-option tracking to finalization, which is deferred to a future version.

MEDIUM

F-10: On-Chain VkZkp1/2/3 Unused — Circuits Not Upgradeable Without Hard Fork

Threat model: Operational (circuit upgrade, bug response).
Affected code:

- `x/vote/types/msgs.go:28--35` (stored)
- `ffi/zkp/halo2/verify.go` (never read)
- `circuits/include/shielded_vote_circuits.h` (VK not parameterized)

Description

`MsgCreateVotingSession` accepts `vk_zkp1`, `vk_zkp2`, `vk_zkp3` from the vote manager and stores them on the `VoteRound` proto. However, no verification function reads these fields. The three Rust FFI verification functions (`sv_verify_delegation_proof`, `sv_verify_vote_proof`, `sv_verify_share_reveal_proof`) accept only `(proof, public_inputs)` — there is no VK parameter. The verification keys are compiled into the static library `libshielded_vote_circuits.a` at build time.

This means:

- ZKP circuits **cannot be upgraded per-round or per-session** — the VK is fixed at compile time.
- If a circuit bug is discovered, remediation requires recompiling the Rust library, rebuilding `svoted`, and coordinating a **hard fork** across all validators simultaneously.
- The on-chain `vk_zkp` fields create a false sense of upgradeability — an operator inspecting the proto definition would assume circuits are swappable when they are not.
- The fields are validated as non-empty (`ValidateBasic`) but never checked for format, size, or correctness.

Recommendation

Either (a) wire the on-chain VKs through to the Rust verifier so circuits are genuinely swappable per-round, or (b) remove the fields entirely to avoid misleading operators. Option (a) enables hot-patching circuits without hard forks.

Remediation

Status: RESOLVED (PR #25 — merged 2026-03-19).

Option (b) was chosen: `vk_zkp1`, `vk_zkp2`, `vk_zkp3` fields removed from both `MsgCreateVotingSession` and `VoteRound` proto messages, along with all `ValidateBasic` checks, keeper field copies, CLI handling, and test fixtures. Proto field numbers were renumbered contiguously rather than marked `reserved` (intentional breaking wire change, acknowledged in PR). This is acceptable for a pre-mainnet chain; any test deployments with persistent state require a full state wipe. The VKs remain derived from the circuit definition and cached inside the Rust FFI binary.

MEDIUM

F-13: RedPallas Identity-Point Signature Bypass — Four Validation Layers Non-Functional

Threat model: External attacker (any party submitting vote transactions).
Affected code:

- `x/vote/ante/validate.go:152, 202` (signature verification)
- `ffi/redpallas/verify_ffi.go:36` (FFI)
- `x/vote/types/msgs.go:67, 142` (`ValidateBasic`)

Description

When a `MsgCastVote` or `MsgDelegateVote` is submitted with the Pallas identity point (`[0x00; 32]`) as the verification key (`r_vpk` or `rk`) and a zero signature (`[0x00; 64]`), the RedPallas signature verification at `validate.go:202/152` **passes silently**. The `reddsa` crate v0.5.1 deliberately allows identity-point verification keys per the Zcash Orchard specification (`verification_key.rs:104--111`). With zero signature and identity key, the Schnorr verification equation $h \cdot (-sB + R + cA) = 0$ collapses: every term is the identity point, and the small-order check passes trivially.

Four independent validation layers fail for this input:

1. **ValidateBasic** (`msgs.go:67/142`) — checks length only (`len == 32`), no curve or identity check.
2. **FFI input validation** (`verify_ffi.go:37`) — checks length only.
3. **reddsa deserialization** — deliberately accepts identity per Orchard spec.
4. **reddsa signature verification** — equation trivially satisfied.

The **sole remaining defense** is `elgamal.UnmarshalPublicKey` inside the ZKP verifier (`verify.go:196/306`), which explicitly checks `pt.IsIdentity()`. This is a boolean comparison in a serialization helper, not a cryptographic gate. No Halo2 proof verification is reached.

Impact

The signature check exists as an independent authentication layer. For identity-point inputs, this layer provides **zero authentication**. For `MsgDelegateVote`, the sighash is client-provided, meaning the identity bypass nullifies all message-content binding from the signature layer. The ZKP prevents actual vote forgery, but any future change weakening or removing the `UnmarshalPublicKey` identity check would leave no functional gate between the attacker and ZKP proof verification.

Additionally, identity-point requests that bypass the signature check force two unnecessary KV store reads (commitment tree root and vote round lookup) before rejection, increasing per-request processing cost compared to the normal signature-rejection path.

Recommendation

Add an explicit identity-point check in `ValidateBasic` for both message types, consistent with the existing 32-byte length checks:

```
var zeroKey [32]byte
if bytes.Equal(msg.RVpk, zeroKey[:]) {
    return fmt.Errorf("r_vpk must not be the identity point")
}
```

This rejects identity-point inputs at the cheapest possible point in the pipeline, before sighash computation, FFI calls, or KV reads.

Remediation

Status: RESOLVED (PR #42 — merged 2026-03-27).

PR #42 adds identity-point rejection at two layers in the Go pipeline:

1. **ValidateBasic:** Both `MsgDelegateVote` and `MsgCastVote` now reject 32 zero bytes for `rk` and `RVpk` respectively, catching identity-point inputs at the cheapest point in the pipeline before sighash computation, FFI calls, or KV reads.
2. **Ante handler:** `elgamal.UnmarshalPublicKey` checks are added before the RedPallas FFI call in both `verifyDelegation` and `verifyCastVote`, rejecting identity and off-curve points as defense-in-depth.

The Go validation pipeline is now self-sufficient against the identity-point signature bypass regardless of Rust-side behavior. Test coverage includes 4 new cases: identity-point and off-curve rejection for both message types.

MEDIUM

F-14: Unbounded Proof Size in MsgRevealShare Enables CPU DoS via FFI

Threat model: Any external party with RPC access (unauthenticated).
Affected code:

- `x/vote/types/msgs.go:162--164` (`ValidateBasic`)
- `x/vote/ante/validate.go:243--267` (`verifyRevealShare`)
- `app/ante.go:112` (infinite gas meter)
- `ffi/zkp/halo2/verify.go:374` (`VerifyShareRevealProof`)
- `circuits/src/ffi.rs:799--804` (Rust FFI entry)

Description

`MsgRevealShare` accepts proof payloads up to ∼500 KB (limited only by CometBFT's `max_tx_bytes = 1 MB` after base64 overhead) and passes them directly to the Halo2 Rust FFI without any size validation. Five validation layers all check for empty proofs but none enforce a maximum:

1. `ValidateBasic` (`msgs.go:162`): `len(proof) == 0` only
2. Ante handler (`validate.go:243--267`): no signature check on `MsgRevealShare`
3. Go FFI bridge (`verify.go:374`): `len(proof) == 0` only
4. Rust FFI entry (`ffi.rs:799`): `proof_len == 0` only
5. Halo2 transcript parser: attempts to deserialize the entire buffer

Unlike `MsgDelegateVote` and `MsgCastVote`, which are gated by RedPallas signature verification at `validate.go:152` and `validate.go:202` respectively, `MsgRevealShare` has **no signature gate** — oversized proofs bypass all cheap validation and reach the Halo2 FFI directly. A valid Halo2 K=14 share-reveal proof is ∼5 KB; the transport allows ∼100× amplification.

Combined with the infinite gas meter applied to all vote transactions (`ante.go:112`), an unauthenticated attacker can submit 200 concurrent 500 KB garbage proofs costing zero gas, pinning a validator's CPU for ∼30 seconds (∼6 block times at 5 s/block). E2E testing on a live 3-validator testnet confirmed all proofs reached the Rust FFI and were rejected at the Halo2 transcript parser. Sustained attack (500 requests across 10 waves) showed 19% latency degradation, indicating memory pressure under load.

Impact

- **CPU denial-of-service:** 200 concurrent 500 KB proofs saturate a validator's CPU for ∼30 s.
- **Zero cost to attacker:** Infinite gas meter; no signature or key material required for `MsgRevealShare`.
- **All node types affected:** Both validators and RPC nodes run `CheckTx` with the full ZKP pipeline.
- **Compounds with F-13:** If the RedPallas identity-point bypass (F-13) is unpatched, all three vote message types reach the FFI, tripling the attack surface.
- **ProcessProposal gap:** Vote txs (tags `0x02--0x04`) are not validated in `ProcessProposal` (`process_proposal.go:74`). A malicious proposer can include oversized-proof vote txs directly in their block proposal, forcing all validators to spend ∼0.2 s per proof in `FinalizeBlock`.

No crashes or OOM observed — Halo2's transcript parser rejects invalid encodings gracefully.

Recommendation

Add a maximum proof size constant and enforce it in `ValidateBasic` for all three vote message types:

```
const MaxProofBytes = 10_000 // valid K=14 proof ~5KB

if len(msg.Proof) > MaxProofBytes {
    return fmt.Errorf("proof␣too␣large:␣%d␣>␣%d",
        len(msg.Proof), MaxProofBytes)
}
```

Apply to `MsgDelegateVote`, `MsgCastVote`, and `MsgRevealShare`. Additionally, add `ValidateBasic` calls in `ProcessProposal` for vote tx tags (`0x02--0x04`) to prevent proposer block-stuffing with oversized proofs.

Remediation

Status: RESOLVED (PR #43 — merged 2026-03-25).

PR #43 adds `MaxProofSize = 8192` and enforces it in `ValidateBasic()` for all three ZKP message types (`MsgDelegateVote`, `MsgCastVote`, `MsgRevealShare`), rejecting oversized proofs before they reach the Rust FFI. Defense-in-depth size checks were also added in the Go FFI bridge (`VerifyDelegationProof`, `VerifyVoteProof`, `VerifyShareRevealProof`). The 8192-byte limit provides ∼57% headroom above the largest valid proof (5,216 bytes for K=14 circuits). Additionally, `MsgRevealShare.ShareNullifier` was tightened from non-empty to exactly 32 bytes, matching the FFI layer requirement. 72 tests pass.

LOW

F-05: Genesis Import Produces Incorrect Tree Root

Threat model: Operational (chain migration via genesis export/import).
Affected code: `x/vote/keeper/genesis.go:25--36`, `x/vote/keeper/keeper.go:114--124`.

Description

`InitGenesis` restores `TreeState` (including `Height > 0`) and commitment leaves (0x02), but does NOT restore shard blobs (0x0F), cap (0x10), or checkpoints (0x11). `ensureTreeLoaded` takes the restart branch, finds an empty shard store, and `Root()` returns the empty-tree root. The EndBlocker silently overwrites `state.Root`, corrupting all subsequent tree roots and invalidating ZKP witnesses.

Recommendation

Set `TreeState.Height = 0` in `InitGenesis` when `NextIndex > 0`, forcing the first-boot replay path that rebuilds the tree from leaves.

Remediation

Status: RESOLVED (PR #45 — merged 2026-03-27).

PR #45 sets `TreeState.Height = 0` in `InitGenesis` when `NextIndex > 0`, forcing the first-boot replay path that rebuilds the `ShardTree` from imported leaves via `AppendFromKV`. Tests include a dedicated `TestInitGenesisClearsTreeHeight` (imports genesis with `Height = 999`, verifies it is zeroed while `NextIndex` and `Root` are preserved) and an updated round-trip test asserting `Height == 0` after import.

LOW

F-09: Genesis Export Omits Partial Decryptions and Block Leaf Index

Threat model: Operational (chain migration).
Affected code: `x/vote/keeper/genesis.go:113--198`.

Description

Genesis export/import omits partial decryptions (0x12) and block leaf index (0x08). If export occurs during TALLYING, validators must re-submit partials. Block leaf index loss is client convenience only.

Recommendation

Add export/import handlers for both prefixes.

Remediation

Status: RESOLVED (PR #47 — merged 2026-03-27).

PR #47 adds genesis export/import handlers for both missing prefixes:

- **Partial decryptions (0x12):** New `GenesisPartialDecryption` proto message with `exportPartialDecryptions()` and corresponding `InitGenesis` restore. If export occurs during TALLYING, partial decryptions are preserved so validators do not need to re-submit.
- **Block leaf indices (0x08):** New `GenesisBlockLeafIndex` proto message exported per round tree and restored on import, preserving the `CommitmentLeaves` query on the migrated chain.

Validation enforces round ID length, validator index ≥ 1, and 32-byte partial decrypt size. Round-trip tests cover multiple validators and multiple block heights, including negative lookups.

LOW

F-11: CommitmentLeaves Query Has No Range Cap — Unbounded Memory Allocation

Threat model: Any unauthenticated RPC client.
Affected code:

- `x/vote/keeper/query_server.go:141--161`
- `x/vote/keeper/keeper_voting.go:153--196`

Description

The `CommitmentLeaves` gRPC query accepts a `[fromHeight, toHeight]` range with no maximum cap. `GetCommitmentLeaves` (`keeper_voting.go:153--196`) iterates all `BlockLeafIndex` entries in the range, allocating a 32-byte leaf per block. A request with `fromHeight=0, toHeight=2^63` forces the node to load the entire commitment tree history into memory (∼720 MB for 10M leaves).

This endpoint is unauthenticated, free (no gas), and has no rate limiting.

Recommendation

Enforce a maximum block range:

```
const MaxCommitmentLeafRange = 1000
if req.ToHeight - req.FromHeight > MaxCommitmentLeafRange {
    return nil, status.Errorf(codes.InvalidArgument, "range␣too␣large")
}
```

This is transparent to normal usage (incremental tree sync polls recent blocks in small ranges).

Remediation

Status: RESOLVED (PR #48 — merged 2026-03-27).

PR #48 adds `MaxCommitmentLeafRange = 1000` in `x/vote/types/keys.go` and rejects requests exceeding the cap with `codes.InvalidArgument` before any KV store iteration. Tests cover both the rejection case (range of 1001) and the boundary (exactly 1000 accepted).

INFO

Informational Findings (I-01 through I-10)

I-01: Non-constant-time point comparison — `bytes.Equal` at `msg_server_tally_decrypt.go:146`. Tally values are public post-finalization; no timing leak.

I-02: Slice aliasing in StripNonAckersFromRound — Reuses backing array via `[:0]`. Intentional and safe.

I-03: Missing ValidateBasic size checks — Several byte fields lack explicit size validation. FFI re-validates via `copyChunk()`. **RESOLVED** (PR #50): exact byte sizes now enforced in `ValidateBasic` for all message fields.

I-04: RedPallas rk not validated as Pallas point — Only checked as 32 bytes before FFI. Rust side validates. **RESOLVED** (PR #42): `elgamal.UnmarshalPublicKey` checks added before RedPallas FFI call, rejecting identity and off-curve points on the Go side.

I-05: UpdateVoteRoundStatus no transition guard — Blindly sets status. Safe today (sole caller is EndBlocker). **RESOLVED** (PR #52): now requires `expectedOldStatus` parameter and returns `ErrInvalidStatusTransition` on mismatch.

I-06: Missing `catch_unwind` on Rust FFI functions — 18 of 19 exported Rust FFI functions in `circuits/src/ffi.rs` lack `catch_unwind`. Only `sv_generate_share_reveal` is wrapped. A Rust panic across the FFI boundary is undefined behavior and would crash the validator process. **RESOLVED** (PR #53): all 19 FFI functions now wrapped in `catch_unwind(AssertUnwindSafe(...))`; new error code `-6` for internal panics; diagnostics via `sv_last_error()`.

I-07: Ack "signature" is not cryptographic — The ceremony ack "signature" at `msg_server_ceremony.go` is `SHA256("ack" || ea_pk || validator_address)`, not a cryptographic signature. Authentication relies on `ValidateProposerIsCreator`, not this hash. The naming is misleading. **RESOLVED** (PR #54): renamed to `AckDigestDomain/sha256AckDigest` with clarifying comments; proto field name preserved for wire compatibility.

I-08: Redundant partial decryption injection when tally threshold met — When the tally threshold is already reached, both the partial decrypt injector and the tally injector fire for the same TALLYING round. The tally tx executes first (prepended last), finalizes the round, and the partial decrypt tx fails cleanly. No security impact; minor gas waste.

I-09: Share file accumulation without cleanup — Share files (`share.<hex(round_id)>`, 32 bytes each, 0600 permissions) at `prepare_proposal_ceremony.go:391` are never deleted after round finalization. These contain Shamir share scalars that are unnecessary post-FINALIZED. **RESOLVED** (PR #49): files are now zero-overwritten, fsynced, and deleted on round finalization; orphaned files cleaned on startup; raw byte buffers zeroed after parsing.

I-10: Dormant single-maintainer Pallas curve dependency — All Go-side elliptic curve operations depend on `github.com/mikelodder7/curvey` v1.1.1 (279 call sites across 13 production files). Extracted from Coinbase's archived `kryptology` by its original author. Pallas field arithmetic is fiat-crypto generated (formally verified constant-time). However: the repo is dormant (last commit Feb 2024), single maintainer, no CI, no security policy. No Go alternative exists for Pallas. Recommend vendoring/forking, and long-term migrating Pallas operations to the Rust FFI layer using `zcash/pasta_curves`.

Trust Model and Security Boundaries

Trust Roles

Role	Trust Level and Capabilities
Vote Manager	Trusted single key. Controls round creation, sets `NcRoot` and `NullifierImtRoot` (ZKP public inputs). A malicious vote manager could craft roots to include fake note commitments or exclude legitimate voters. The chain has no mechanism to independently verify these roots.
Dealing Proposer	Sees full `ea_sk` in Step 1 (Finding F-03). Must be honest for individual vote confidentiality. A DKG protocol would eliminate this trust assumption.
Validators	Must maintain share files on disk. $\geq t$ honest validators needed for tally liveness (Finding F-04). Bonded validator status required for ceremony participation. Permissioned set controlled by VoteManager.
Voters	Zero-knowledge — no trust required beyond possessing valid credentials and the vote manager having provided honest roots.
External Parties	Should have no ability to modify chain state. The noop-signer bypass (F-01) required a funded on-chain account (i.e., a validator or vote manager) and did not extend to external parties; resolved via PRs #15, #16 (ante handler message-type blocking).

Security Properties

Property	Description	Status
Vote privacy	Individual votes are encrypted under `ea_pk`; only aggregate totals are revealed at finalization.	Holds[1]
Vote integrity	Only ZKP-authenticated votes modify the tally accumulator.	Holds[2]
Tally correctness	Lagrange combination of $\geq t$ DLEQ-verified partial decryptions recovers the correct aggregate.	Holds
Nullifier uniqueness	Each voter can only vote once per round; replays are rejected.	Holds
Proposer injection	Auto-injected messages (Deal, Ack, PartialDecrypt, Tally) cannot be submitted via the mempool.	Holds[3]
Ceremony freshness	Each ceremony uses a fresh `ea_sk`; ack hashes are bound to `ea_pk`.	Holds
Tree determinism	All validators compute identical Poseidon roots from identical KV state.	Holds

[1]Conditional on an honest dealer (Finding F-03). Share counts leak distribution (Finding F-08).
[2]Previously broken by F-01 (noop-signer bypass, resolved PRs #15, #16) and F-12 (sign-bit flip, resolved PR #26). Both tally corruption vectors are now closed.
[3]F-01 fix ensures both tx paths are now gated.

Compound Risk: F-03 + F-08 Privacy Degradation

Findings F-03 (malicious dealer retains `ea_sk`) and F-08 (share counts leak per-option distribution) are each classified as **Accepted Risk**. Their compound effect exceeds the sum of parts:

1. A rogue dealer retains `ea_sk` and can decrypt any individual voter's ciphertext: $v \cdot G = C_2 - \texttt{ea_sk} \cdot C_1$.
2. Per-option share counts are observable in real time through three channels: KV prefix `0x0B` (raw ABCI query), `reveal_share` event attributes (`proposal_id`, `vote_decision`), and the `VoteSummary` gRPC query.
3. The dealer observes a count increment for option k at block height h, then decrypts the corresponding ciphertext to confirm or narrow down voter identity.

Mitigation — helper server timing obfuscation. The helper submission server implements three layers of timing decorrelation that substantially reduce the practical exploitability of this attack chain:

- **Enqueue delay:** Each share receives a random delay from [`MinDelay`, `vote_end_time` − 60s) using `crypto/rand` (default floor: 90s), decoupling wallet submission time from on-chain appearance.
- **Poisson processing:** The processor wakes at exponentially distributed intervals (Exp(1/`ProcessInterval`), default mean 30s) and adds intra-share exponential jitter between submissions within each batch. On-chain submission order does not reflect the order shares became ready.
- **Crash recovery:** All in-memory scheduling is re-randomized on restart. No timing data persists to disk.

Residual exposure: The 60-second deadline cutoff skips jitter for shares near `vote_end_time` to ensure liveness. Shares submitted in the final minute are distinguishable as a timing class. For small voter populations (< 10 per option), the combination of real-time distribution counts and decryption capability may still enable probabilistic deanonymization even with timing obfuscation. Deployments with strict per-voter privacy requirements should prioritize resolving F-03 (DKG) and F-08 (deferred count tracking).

Codebase Maturity Evaluation

The following assessment rates the vote-sdk codebase across five dimensions using a Weak / Moderate / Satisfactory / Strong scale.

Dimension	Rating	Key Evidence
Testing	Satisfactory	50 test files, 331 functions, 19,263 LOC; no fuzz or coverage tracking
Documentation	Strong	Ceremony flow, state machine, deployment guide, KV layout all documented
Code Quality	Strong	Zero `_ = err`; consistent error wrapping; secret zeroing with `defer`
CI/CD	Satisfactory	5 workflows (lint, unit, integration, Halo2, stress); race detector on
Dependency Hygiene	Strong	All Rust git deps pinned to exact commits; `go.sum` and `Cargo.lock` committed

Testing — Satisfactory. The codebase contains 50 test files with 331 test functions spanning 19,263 lines of test code. Tests cover all critical paths: keeper operations, ante handler validation, cryptographic primitives (ElGamal, ECIES, Shamir, DLEQ), FFI boundary verification (Halo2, RedPallas), ABCI integration, and helper server operations. Negative-case coverage is strong: 31 test files explicitly test error conditions including invalid inputs, cryptographic failures, state machine violations, and attack scenarios. The validation ordering test (`TestValidateVoteTx_ValidationOrder`) verifies that the pipeline rejects messages at the correct stage. Gaps: no fuzz testing (high-value for ZKP deserialization and wire format parsing), no benchmarks, and no coverage tracking.

Documentation — Strong. The project includes a 336-line README covering architecture, wire format tags, KV store layout, and API endpoints. Dedicated documentation covers the TSS ceremony workflow (`tss-ceremony.md`, 204 lines), a hand-verified mathematical walkthrough in $\mathbb{F}_{23}$ (`tss-ceremony-math.md`, 439 lines), the session status state machine with Mermaid diagrams (`session-status-lifecycle.md`), and a deployment guide with systemd units (`deploy-setup.md`). Critical source files are well-commented, particularly the ceremony handlers (75+ inline comments).

Code Quality — Strong. Error handling is exemplary: zero `_ = err` patterns in production code. All KV store operations, FFI calls, and crypto operations consistently check and propagate errors with context wrapping. Secret zeroing via dedicated functions (`zeroBytes`, `zeroScalar`) with `defer` for guaranteed cleanup. The primary concern is 6 `panic()` calls in production runtime paths (crypto/shamir, crypto/elgamal, prepare_proposal), though triggering conditions are theoretically unreachable for valid inputs.

CI/CD — Satisfactory. Five GitHub Actions workflows cover linting, unit tests, integration tests, Halo2 FFI verification, and deterministic stress testing. Race detector enabled for unit and integration tests. Multi-platform release pipeline (Linux/macOS amd64/arm64). Gaps: minimal linting (`go vet` only, no `golangci-lint`), no coverage tracking, no security-focused static analysis (`gosec`, `govulncheck`).

Dependency Hygiene — Strong. Go dependencies (23 direct) sourced from canonical repositories. Three replace directives documented with security rationale. Rust dependencies include 4 valargroup-maintained git dependencies, all pinned to exact commit hashes. `Cargo.lock` committed for reproducible builds. Main gap: no automated vulnerability scanning in CI.

Recommendations

Recommendations are ordered by priority (CRITICAL first).

Immediate (Pre-Launch)

1. **[F-01] Add standard-chain message-type rejection decorator.** **RESOLVED** (PRs #15, #16). Two-layer defense: multi-message tx rejection + per-type vote/ceremony message blocking at the ante handler. 27 tests cover all attack vectors including tally corruption repro.
2. **[F-02] Reject MsgRevealShare during TALLYING.** **RESOLVED** (PR #21). `AcceptsTallyingRound()` returns `false`; `ValidateRoundForShares` removed as dead code.
3. **[F-12] Expose full (x, y) coordinates in ZKP #3 circuit.** **RESOLVED** (PR #26). Circuit updated to bind full curve point representation (7 → 9 public inputs), preventing sign-bit tally corruption.
4. **[F-08] Close all three vote-distribution leak channels.** **ACCEPTED RISK** — deferred to future version. (a) Remove per-decision share count tracking during active rounds; defer to finalization. (b) Strip `proposal_id`, `vote_decision`, and `share_nullifier` from the `reveal_share` event. (c) Gate `VoteSummary` `BallotCount` to return zero for non-finalized rounds.

Short-Term

1. **[F-04] Add TALLYING timeout to EndBlocker.** **RESOLVED** (PR #44). Configurable timeout (default 6 hours); on expiry, round transitions to FINALIZED with `tally_timed_out = true`.
2. **[F-05] Fix genesis import tree root.** **RESOLVED** (PR #45). `TreeState.Height` zeroed in `InitGenesis` to force first-boot replay.
3. **[F-06] Add message-type allowlist for standard Cosmos txs.** **RESOLVED** (PR #17). `MsgSend`/`MsgMultiSend` blocked; replaced with role-based `MsgAuthorizedSend` restricting transfers to vote manager and bonded validators.
4. **[F-07] Clear helper witness data on submission.** **RESOLVED** (PR #46). Witness columns zeroed on submit/permanent failure; `PRAGMA secure_delete=ON` enabled.
5. **[F-09] Add genesis handlers for 0x12 and 0x08 prefixes.** **RESOLVED** (PR #47). Partial decryptions and block leaf indices now exported/imported with validation.
6. **[F-11] Add range cap to CommitmentLeaves query.** **RESOLVED** (PR #48). `MaxCommitmentLeafRange = 1000` enforced before KV iteration.
7. **[F-13] Reject identity-point verification keys in ValidateBasic.** **RESOLVED** (PR #42). Identity-point and off-curve rejection added at both `ValidateBasic` and ante handler layers for `MsgCastVote` and `MsgDelegateVote`.
8. **[F-14] Add maximum proof size to ValidateBasic.** **RESOLVED** (PR #43). `MaxProofSize = 8192` enforced in `ValidateBasic` for all three ZKP message types; defense-in-depth size checks added in the Go FFI bridge.
9. **[F-15] Enforce threshold policy on-chain.** **RESOLVED** (PR #40). `ThresholdForN(n)` enforced as exact match in both `validateInjectedDeal` and the keeper handler; tests at unit, handler, and ABCI layers.

10. **[F-10] Wire on-chain VKs to the Rust verifier or remove the fields.** **RESOLVED** (PR #25). Fields removed from proto, validation, keeper, CLI, and tests.

Long-Term

1. **[F-03] Replace single-dealer with DKG.** **ACCEPTED RISK** — deferred to future version. Implement a distributed key generation protocol so no single validator knows the full `ea_sk`.
2. **[I-03, I-04] Add defense-in-depth ValidateBasic size checks and RedPallas point validation.** **RESOLVED** (PRs #50, #42). Exact byte sizes enforced in `ValidateBasic`; `rk`/`RVpk` validated as Pallas curve points before FFI.
3. **[I-05] Add transition guard to UpdateVoteRoundStatus.** **RESOLVED** (PR #52). Now requires `expectedOldStatus` parameter.

Conclusion

The Shielded Vote SDK demonstrates strong cryptographic design and careful implementation across most of its codebase. The custom wire format, threshold ceremony, Poseidon Merkle tree FFI, and ZKP validation pipeline are all correctly implemented. The protocol's privacy and integrity guarantees hold when messages traverse their intended code paths.

The audit identified one **CRITICAL architectural vulnerability** (F-01) in the dual-mode transaction routing that allowed vote messages to bypass the entire ZKP verification pipeline via standard Cosmos Tx envelopes. This finding was **confirmed exploitable** through static analysis and has since been **resolved** (PRs #15, #16, merged 2026-03-18) with a two-layer ante handler defense.

Of the four HIGH findings:

- F-02 (late share race) has been **resolved** (PR #21) by rejecting `MsgRevealShare` during TALLYING.
- F-03 (malicious dealer) is an **accepted risk** — the permissioned validator set and round-robin dealer rotation provide partial mitigation, and a full DKG protocol is deferred to a future version.
- F-12 (sign-bit tally corruption) has been **resolved** (PR #26) by updating the circuit to bind the full curve point representation ($7 \rightarrow 9$ public inputs).
- F-15 (threshold ceremony downgrade) has been **resolved** (PR #40) by enforcing `ThresholdForN(n)` as an exact match in both the keeper handler and `ProcessProposal`.

Combined with the F-01 (CRITICAL) fix, the tally pipeline is now protected against both external and internal accumulator corruption vectors.

Of the seven MEDIUM findings, six have been **resolved** (F-04 via PR #44, F-06 via PR #17, F-07 via PR #46, F-10 via PR #25, F-13 via PR #42, F-14 via PR #43) and one is **accepted risk** (F-08). All three LOW findings have been **resolved**: F-05 (genesis tree root corruption, PR #45), F-09 (genesis prefix omission, PR #47), and F-11 (unbounded query, PR #48). Ten INFO findings round out the report.

All findings have been either resolved or accepted as risk. No open findings remain.

Overall assessment: The codebase is well-engineered with strong security primitives. The CRITICAL finding and three of four HIGH findings have been resolved; one HIGH is accepted risk (F-03). Thirteen findings have been resolved in total. The system provides a robust foundation for private on-chain voting.